

# OpenFOAM: Primeiras Simulações

## CFD: Curso Básico

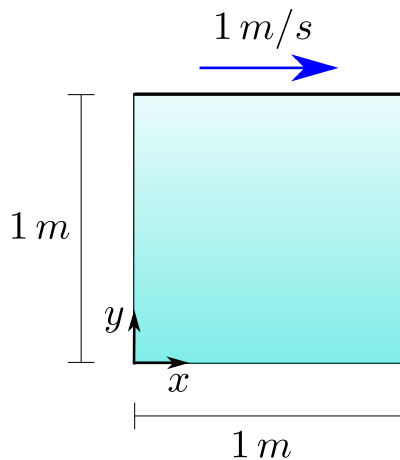
Professor: Adriano Possebon Rosa

Departamento de Engenharia Mecânica  
Universidade de Brasília

# Sumário

- 1 Cavidade Cisalhante
- 2 Escoamento Entre Placas
- 3 Cavidade Cisalhante 3D
- 4 Cavidade Cisalhante com Fluido Não Newtoniano
- 5 Conclusão

Vamos começar nossos estudos em CFD com o problema da cavidade cisalhante.



Neste caso, vamos simular o comportamento de um fluido newtoniano e incompressível em uma cavidade quadrada de lado  $1\text{ m}$ .

As paredes da esquerda, da direita e de baixo estão paradas. A parede de cima se move para a direita com uma velocidade de  $1\text{ m/s}$ .

Vamos considerar que a viscosidade cinemática do fluido  $\nu$  é igual a  $0.01\text{ m}^2/\text{s}$ .

Resolver esse problema significa encontrar o valor da pressão  $p$  e da velocidade  $\mathbf{U}$  em cada ponto do domínio e em cada instante de tempo.

Vamos resolver a equação da conservação de massa,

$$\nabla \cdot \mathbf{U} = 0 ,$$

e a equação de Navier-Stokes,

$$\frac{\partial \mathbf{U}}{\partial t} + \nabla \cdot (\mathbf{U}\mathbf{U}) = -\nabla \left( \frac{P}{\rho} \right) + \nabla \cdot (\nu \nabla \mathbf{U}) ,$$

considerando o escoamento incompressível.

Vamos começar.

Primeiro, copie a pasta

*openfoam11/tutorials/incompressibleFluid/cavity*

para uma pasta de sua preferência.

No OpenFOAM, cada simulação é uma pasta.

Dê o nome de *cavidadeCisalhante* a essa pasta.

Temos 3 pastas (*folders* ou *directories*) dentro da pasta *cavidadeCisalhante*:

- 0
- constant
- system

Essa é a estrutura básica para todas as simulações no OpenFOAM.

Na pasta **0 (ZERO)**, temos as condições iniciais e de contorno para todas as variáveis primitivas que estamos resolvendo.

Na pasta **constant** nós definimos as propriedades físicas da simulação (viscosidade, densidade, modelos de turbulência, etc.)

Na pasta **system** nós temos os parâmetros que controlam a simulação (tempo inicial, passo de tempo, tempo final, ...), os modelos numéricos e de convergência e outras definições numéricas do caso.



EM CFD, toda simulação é dividida em 3 etapas: **pré-processamento**, **processamento** e **pós-processamento**.

**Pré-processamento:** preparação dos dados e da configuração do problema (geometria, malha, condições iniciais e condições de contorno).

**Processamento:** resolução numérica das equações.

**Pós-processamento:** análise e interpretação dos resultados.

Vamos começar com a geração da malha.

Neste exemplo da cavidade cisalhante vamos usar o software *blockMesh*, que já vem instalado com o OpenFOAM, para gerar a malha.

Para isso vamos dar uma olhada no arquivo *system/blockMeshDict*

*Dict* remete a dicionário, é uma forma de armazenar informações.

## *system/blockMeshDict*

```
1  convertToMeters 1;
2
3  vertices
4  (
5      (0 0 0)      // 0
6      (1 0 0)      // 1
7      (1 1 0)      // 2
8      (0 1 0)      // 3
9      (0 0 0.1)    // 4
10     (1 0 0.1)    // 5
11     (1 1 0.1)    // 6
12     (0 1 0.1)    // 7
13 );
14
15 blocks
16 (
17     hex (0 1 2 3 4 5 6 7) (20 20 1) simpleGrading (1 1 1)
18 );
```

```

19 boundary
20 (
21     movingWall
22     {
23         type wall;
24         faces
25         (
26             (3 7 6 2)
27         );
28     }
29     fixedWalls
30     {
31         type wall;
32         faces
33         (
34             (0 4 7 3)
35             (2 6 5 1)
36             (1 5 4 0)
37         );
38     }

```

```

39     frontAndBack
40     {
41         type empty;
42         faces
43         (
44             (0 3 2 1)
45             (4 5 6 7)
46         );
47     }
48 );

```

Note que eu fiz algumas mudanças na nossa simulação com relação ao caso original.

Deixe o seu caso igual ao que está sendo apresentado aqui.

No arquivo *blockMeshDict*, eu mudei apenas o valor de *convertToMeters* para 1.

O *convertToMeters* é um fator de escala dos valores que estão apresentados no arquivo para os valores que realmente serão gerados. Vamos deixar como 1 mesmo.

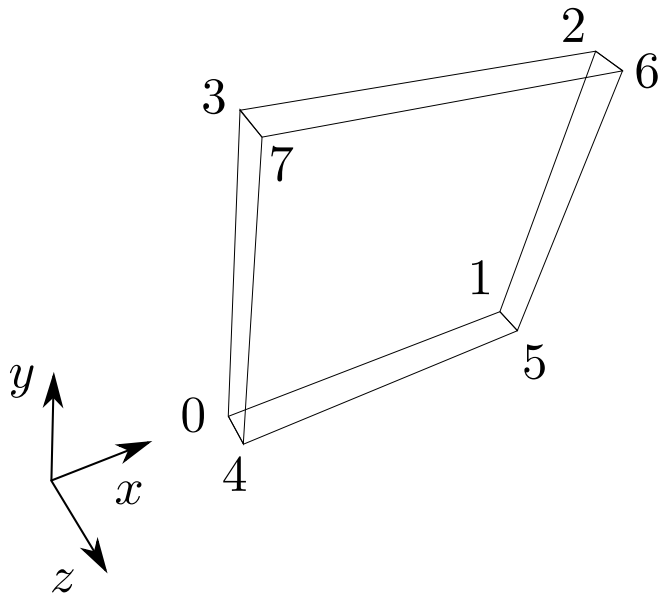
Depois do *convertToMeters*, nós temos a lista de vértices da nossa geometria. O número na frente de cada vértice é um comentário apenas para identificar o vértice.

Tudo que vem depois das duas barras *//* em uma linha não é executado pelo OpenFOAM. Esses comentários nos ajudam na hora de ler o código.

Queremos simular um quadrado, ou seja, uma simulação bidimensional. No entanto, todas as simulações no OpenFOAM são tridimensionais.

Por isso, vamos criar um paralelepípedo (3D) e vamos informar que não queremos resolver o problema em uma direção.

A distância na direção  $z$  não é importante. Estamos usando  $0.1\text{ m}$  aqui.



Com a lista de vértices, nós vamos criar um bloco de malha, ou um *block*.

Esse bloco é feito de hexaedros, com 20 células na direção  $x$ , 20 na direção  $y$  e apenas 1 na direção  $z$ .

O parâmetro *simpleGrading* apresenta uma razão para o tamanho das células. Vamos deixá-lo como está.

Depois, mude o valor do *simpleGrading* para 1.2 e 0.8 e veja o que acontece.



Na sequência vem uma lista importantíssima que é a definição dos tipos de **condições de contorno** que vamos empregar em cada superfície.

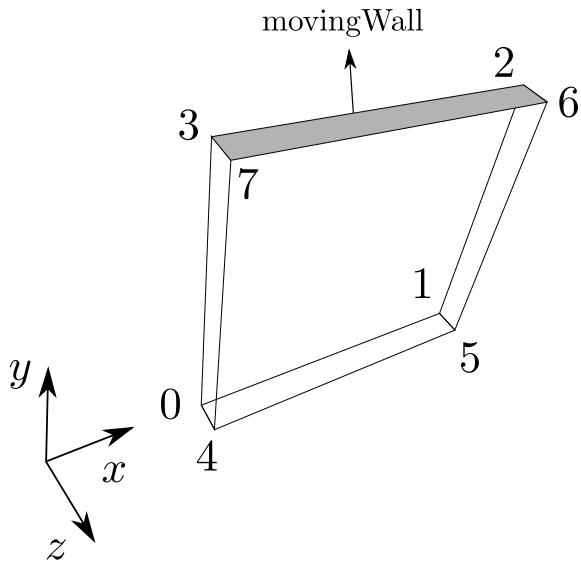
Todas as superfícies precisam estar em uma dos grupos definidos.

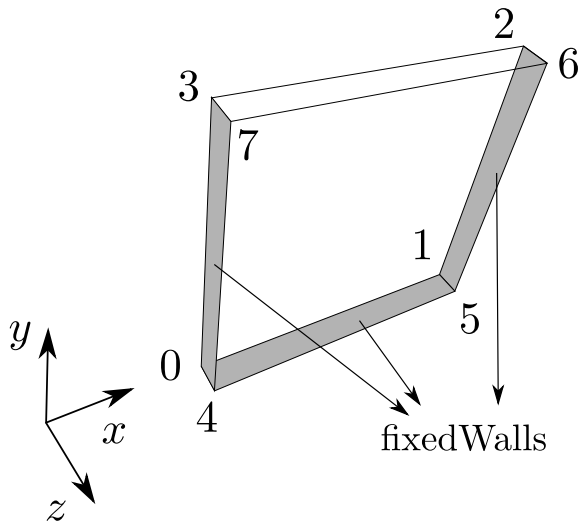
Neste caso, temos 3 grupos de superfícies de contorno: *movingWall*, *fixedWalls* e *frontAndBack*.

*movingWall* e *fixedWalls* são do tipo *wall*, enquanto *frontAndBack* são do tipo *empty*.

O tipo *empty* indica que não queremos resolver o problema na direção perpendicular a essas superfícies.

Tenha muita calma na hora de definir as condições de contorno do seu problema. Este é um ponto importante da simulação e interfere muito no resultado final.





## Tudo certo!!! Vamos rodar.

O comando que deve ser executado no terminal para gerar a malha com o *blockMeshDict* é o

```
$: blockMesh
```

Depois de executar o comando, a pasta ***constant/polyMesh*** é criada e todas as informações da malha são adicionadas lá.

Dê uma olhada com calma nos arquivos criados. No próximo slide vamos ver o *boundary* .

## *constant/polyMesh/boundary*

```
1 3
2 (
3   movingWall
4   {
5     type          wall;
6     inGroups      List<word> 1(wall);
7     nFaces        20;
8     startFace     760;
9   }
10  fixedWalls
11  {
12    type          wall;
13    inGroups      List<word> 1(wall);
14    nFaces        60;
15    startFace     780;
16  }
```

```
17     frontAndBack
18     {
19         type            empty;
20         inGroups        List<word> 1(empty);
21         nFaces          800;
22         startFace       840;
23     }
24 )
```

Para verificar se a malha está ok, execute o comando

**\$: checkMesh**

e veja as informações em tela. Iremos interpretar essas informações na aula sobre malhas.

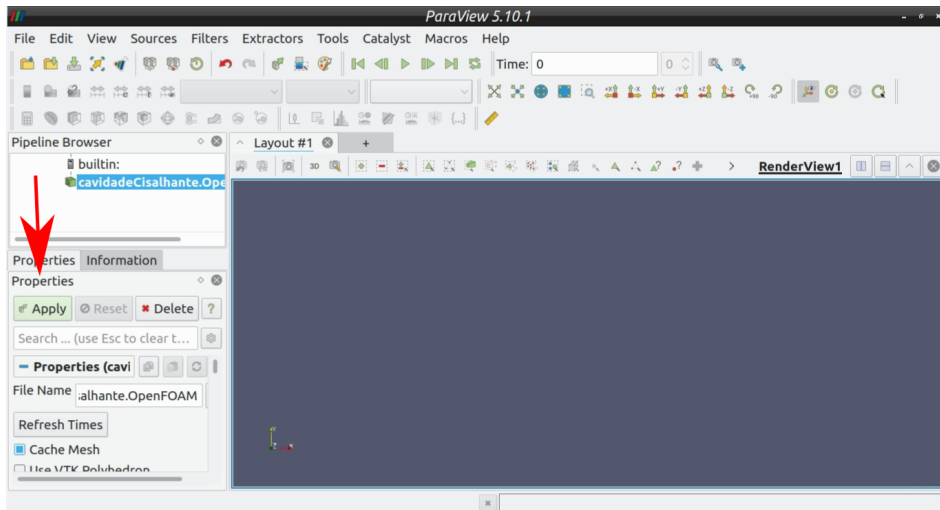
Podemos também visualizar a malha gerada. Execute o comando

**\$: paraFoam**

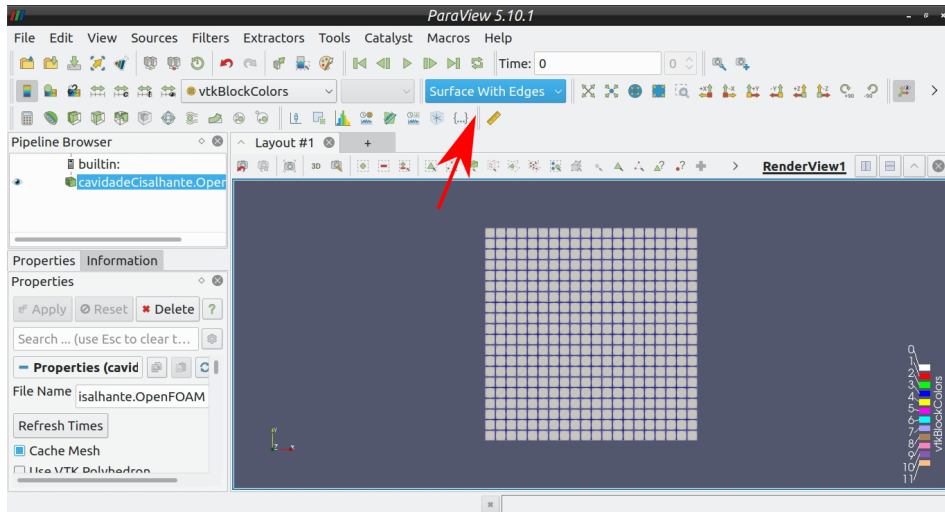
O *ParaView* vai abrir. Clique em *Apply* e depois mude a opção de *Surface* para *Surface With Edges*.

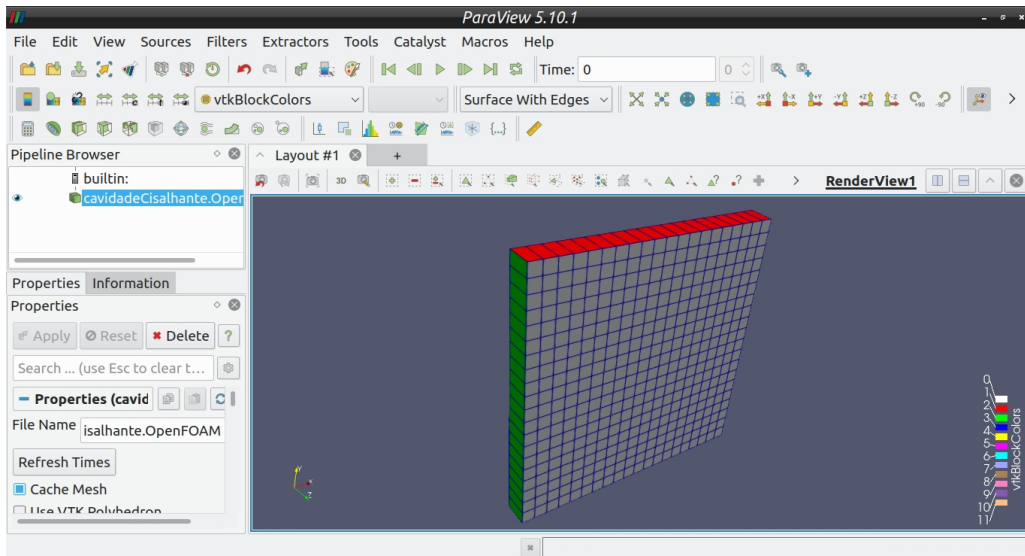


A seta indica o botão *Apply*, que está no lado esquerdo da tela, na altura central.



A seta indica o quadro com as opções de visualização. O quadro está no centro e na parte de cima.





Pronto. Tudo certo com a malha. Agora vamos para as propriedades físicas.

Na pasta *constant* temos dois arquivos, o *momentumTransport* e o *physicalProperties*.

No arquivo *momentumTransport* nós vamos definir o modelo de turbulência que vamos utilizar e também o modelo de viscosidade. Nossa simulação é laminar e nosso fluido é newtoniano (que já é o padrão).

## *constant/momentumTransport*

```
1 FoamFile
2 {
3     format      ascii;
4     class       dictionary;
5     location    "constant";
6     object      momentumTransport;
7 }
8 // * * * * *
9
10 simulationType laminar;
```

No arquivo *physicalProperties* nós vamos definir o valor da viscosidade cinemática  $\nu$  (nu).

A viscosidade cinemática é definida como sendo a razão entre a viscosidade dinâmica  $\mu$  e a densidade do fluido  $\rho$ , ou seja,

$$\nu = \frac{\mu}{\rho} .$$

O código no arquivo *physicalProperties* é apresentado no próximo slide.

## *constant/physicalProperties*

```
1 FoamFile
2 {
3     format      ascii;
4     class       dictionary;
5     location     "constant";
6     object       physicalProperties;
7 }
8 // * * * * *
9
10 viscosityModel constant;
11
12 nu              [0 2 -1 0 0 0 0] 0.01;
13
14 // *****
```

Primeiro, temos a definição de *viscosityModel* como sendo *constant*. Essa simulação é para um fluido newtoniano, que possui viscosidade constante.

Depois temos a definição do valor da viscosidade *nu*.

O valor de *nu* neste caso é  $0.01 \text{ m}^2/\text{s}$ .

Todas as variáveis no OpenFOAM são dimensionais.

Sempre que uma variável é introduzida, temos que definir sua dimensão e seu valor.



As unidades da viscosidade aparecem entre colchetes.

As unidades, no SI, são apresentadas da seguinte forma:

$$[kg \ m \ s \ K \ mol \ A \ cd]$$

Essas unidades representam as 7 dimensões básicas: massa, comprimento, tempo, temperatura, quantidade, corrente e intensidade luminosa.

A unidade da viscosidade é

$$[0 \ 2 \ -1 \ 0 \ 0 \ 0 \ 0]$$

que corresponde a  $m^2/s$ .

O valor da viscosidade é  $0.01 \text{ m}^2/\text{s}$ . O parâmetro governante deste problema é o número de Reynolds  $Re$ , definido como

$$Re = \frac{UL}{\nu},$$

em que  $U$  é a velocidade da parede de cima e  $L$  é o tamanho da cavidade.

Como  $U = 1 \text{ m/s}$  (vamos definir daqui a pouco) e  $L = 1 \text{ m}$ , o número de Reynolds do nosso problema é

$$Re = \frac{1 \times 1}{0.01} = 100$$

Definidas as propriedades do fluido e o modelo de turbulência utilizado (laminar, no caso), vamos para as condições iniciais e de contorno.

## Pasta 0 (ZERO).

Pasta com as condições iniciais e de contorno de todas as variáveis primitivas.

Como a nossa simulação é laminar, estamos resolvendo apenas  $U$  (velocidade) e  $p$  (pressão). Apague os outros arquivos na pasta 0, deixando apenas os arquivos  $U$  e  $p$ . (Esses outros arquivos são para variáveis relacionadas aos modelos de turbulência.)

Vamos começar com a velocidade  $U$ .

$0/U$ 

```

1 FoamFile
2 {
3     format      ascii;
4     class       volVectorField;
5     object      U;
6 }
7
8 dimensions      [0 1 -1 0 0 0 0];
9
10 internalField uniform (0 0 0);
11
12 boundaryField
13 {
14     movingWall
15     {
16         type      fixedValue;
17         value      uniform (1 0 0);
18     }

```

```

19     fixedWalls
20     {
21         type      noSlip;
22     }
23
24     frontAndBack
25     {
26         type      empty;
27     }
28 }

```

O campo de velocidade é um *volVectorField*, ou seja, um campo vetorial com valores no centro das células.

A dimensão da velocidade é  $m/s$ , ou seja,

$$[0 \ 1 \ -1 \ 0 \ 0 \ 0 \ 0]$$

A variável *internalField* indica o valor inicial da variável. Neste caso o valor vai ser 0, uniformemente, em todo o domínio.

Lembre-se de que a velocidade é um vetor. Por isso usamos  $(0 \ 0 \ 0)$ .

Na sequência temos um dicionário chamado *boundaryField*. Temos que apresentar informações sobre todas as superfícies do nosso domínio.

A condição de contorno para  $U$  da superfície do grupo *movingWall* é do tipo *fixedValue*, com valor igual a  $(1 \ 0 \ 0)$ .

Isso é a velocidade na parede de cima. Estamos dizendo que essa parede tem velocidade de  $1 \text{ m/s}$  na direção  $x$  e zero nas outras direções.

Para as superfícies que estão designadas como *fixedWalls*, estamos definindo que a velocidade é sempre zero no contato com elas. Essa é a condição de não deslizamento, ou *noSlip*.

Essa condição é muito utilizada em paredes.

Por fim, como o nosso problema é bidimensional, vamos informar que a condição de contorno para as superfícies *frontAndBack* são *empty*, ou seja, não vamos resolver o problema na direção perpendicular a essas paredes (direção  $z$ ).

Agora é a vez da **pressão**.

$0/p$ 

```

1 FoamFile
2 {
3     format      ascii;
4     class       volScalarField;
5     object      p;
6 }
7
8 dimensions      [0 2 -2 0 0 0 0];
9
10 internalField uniform 0;
11
12 boundaryField
13 {
14     movingWall
15     {
16         type      zeroGradient;
17     }

```

```

18     fixedWalls
19     {
20         type      zeroGradient;
21     }
22     frontAndBack
23     {
24         type      empty;
25     }
26 }

```



A pressão é um *volScalarField*: um campo escalar.

Todos os solvers incompressíveis do OpenFOAM usam a pressão modificada, que é a pressão dividida pela densidade:

$$p = \frac{P}{\rho}$$

A dimensão da pressão modificada é  $m^2/s^2$ , ou

$$[0 \ 2 \ -2 \ 0 \ 0 \ 0 \ 0]$$

O valor inicial da pressão na cavidade é 0. Essa é a pressão relativa. Como estamos resolvendo um escoamento incompressível, o que importa é a variação da pressão, e não o seu valor absoluto.

A condição de contorno mais usada para a pressão em paredes sólidas é a condição de gradiente nulo na direção normal, ou *zeroGradient*.

Essa condição é aplicada para a parede de cima que está se movendo, a *movingWall*, e para as paredes que estão paradas, as *fixedWalls*.

Para as superfícies *frontAndBack* implementamos a condição *empty* novamente.

Já temos a malha computacional, as propriedades físicas da simulação e as condições iniciais e de contorno.

Vamos agora para a parte operacional da simulação, com os parâmetros numéricos. Esses parâmetros podem ser encontrados na pasta ***system***.

O *controlDict* é o arquivo com as instruções gerais sobre a simulação.

# *system/controlDict*

```
1 FoamFile
2 {
3     format      ascii;
4     class       dictionary;
5     location    "system";
6     object      controlDict;
7 }
8 // * * * * *
9
10 application    foamRun;
11
12 solver         incompressibleFluid;
13
14 startFrom      startTime;
15
16 startTime      0;
17
18 stopAt         endTime;
19
20 endTime        1.0;
```

```
21 deltaT      0.01;  
22  
23 writeControl adjustableRunTime;  
24  
25 writeInterval 0.1;  
26  
27 purgeWrite   0;  
28  
29 writeFormat   ascii;  
30  
31 writePrecision 6;  
32  
33 writeCompression off;  
34  
35 timeFormat     general;  
36  
37 timePrecision  6;  
38  
39 runTimeModifiable true;
```

Nesta simulação estamos usando a aplicação *foamRun* com o solver *incompressibleFluid*.

A simulação começa a partir do *startTime*, que está definido como sendo 0.

A simulação vai para no tempo *endTime*, que está definido como 1.0.

O passo de tempo utilizado nesta simulação ( $\Delta t$ ) está definido como sendo 0.01 s, na variável *deltaT*.

Assim, teremos 100 passos de tempo na simulação completa.

Depois do  $\Delta T$ , temos alguns parâmetros que vão definir os arquivos de saída.

O *writeControl* está definido como *adjustableRunTime*. Nesse caso, o passo de tempo será adaptado para que a saída da simulação seja apresentada a cada intervalo de tempo, definido em *writeInterval*.

No nosso caso, queremos escrever os resultados a cada  $0.1\text{ s}$ . Como a nossa simulação vai de 0 a  $1\text{ s}$ , teremos 10 pastas com as soluções (0, 0.1, 0.2, ...).

Vamos deixar os outros parâmetros (*purgeWrite*, *writeFormat*, *writePrecision*, *writeCompression*, *timeFormat*, *timePrecision* e *runTimeModifiable*) como estão.

Voltaremos a discutir o arquivo *controlDict* em outras aulas de nosso curso.



Na pasta ***system***, nós temos também os arquivos *fvSchemes* e *fvSolution*.

O *fvSchemes* define os esquemas de discretização de volumes finitos, ou seja, como são realizadas as integrações no tempo e no espaço.

Já o *fvSolution* apresenta os parâmetros do algoritmo de simulação, os métodos de solução dos sistemas lineares e as tolerâncias utilizadas.

Note que os arquivos estão um pouco diferentes do caso original. Mas não tem problema.

Não vamos ver os detalhes agora. Vamos voltar nesses arquivos nas próximas aulas.

## *system/fvSchemes*

```

1  ddtSchemes
2  {
3      default      Euler;
4  }
5
6  gradSchemes
7  {
8      default      Gauss linear;
9      grad(p)      Gauss linear;
10 }
11
12 divSchemes
13 {
14     default      none;
15     div(phi,U)   Gauss linear;
16 }

```

```

17 laplacianSchemes
18 {
19     default      Gauss linear orthogonal;
20 }
21
22 interpolationSchemes
23 {
24     default      linear;
25 }
26
27 snGradSchemes
28 {
29     default      orthogonal;
30 }

```

# system/fvSolution

```

1 solvers
2 {
3     p
4     {
5         solver      PCG;
6         preconditioner DIC;
7         tolerance    1e-06;
8         relTol        0.05;
9     }
10
11    pFinal
12    {
13        $p;
14        relTol        0;
15    }

```

```

16
17    "(U|UFinal)"
18    {
19        solver      smoothSolver;
20        smoother     symGaussSeidel;
21        tolerance    1e-05;
22        relTol        0;
23    }
24 }
25
26 PIMPLE
27 {
28     nCorrectors      2;
29     nNonOrthogonalCorrectors 0;
30     pRefCell          0;
31     pRefValue          0;
32 }

```

Tudo verificado! Agora sim podemos fazer os cálculos.

**Para rodar a simulação**, execute o comando

**\$: foamRun**

Vão aparecer várias informações sobre a simulação na tela. Elas nos dizem como está a simulação e se algo está fora do esperado.

```
// * * * * *  
Create time  
  
Create mesh for time = 0  
  
Selecting solver incompressibleFluid  
Selecting viscosity model constant  
Selecting turbulence model type laminar  
Selecting laminar stress model Stokes  
No MRF models present  
  
Courant Number mean: 0 max: 0  
  
PIMPLE: No convergence criteria found  
  
PIMPLE: Operating solver in transient mode with 1 outer corrector  
PIMPLE: Operating solver in PISO mode  
  
Starting time loop
```

```
Courant Number mean: 0 max: 0  
Time = 0.001s
```

```
smoothSolver: Solving for Ux, Initial residual = 1, Final residual = 7.86019e-06, No Iterations 13  
smoothSolver: Solving for Uy, Initial residual = 0.990973, Final residual = 7.08708e-06, No Iterations 11  
GAMG: Solving for p, Initial residual = 1, Final residual = 0.0300802, No Iterations 3  
time step continuity errors : sum local = 1.64727e-05, global = 1.74701e-20  
GAMG: Solving for p, Initial residual = 0.326659, Final residual = 4.67831e-07, No Iterations 13  
time step continuity errors : sum local = 4.9902e-10, global = 1.4426e-20, cumulative = 1.4426e-20  
ExecutionTime = 0.049312 s ClockTime = 0 s
```

```
Courant Number mean: 0.0427361 max: 0.169311  
Time = 0.05s
```

```
smoothSolver: Solving for Ux, Initial residual = 0.000790653, Final residual = 5.11792e-06, No Iterations 6  
smoothSolver: Solving for Uy, Initial residual = 0.00119873, Final residual = 7.88519e-06, No Iterations 6  
GAMG: Solving for p, Initial residual = 0.000489861, Final residual = 3.24376e-05, No Iterations 2  
time step continuity errors : sum local = 3.91778e-08, global = -4.9879e-20  
GAMG: Solving for p, Initial residual = 0.00021758, Final residual = 9.09843e-07, No Iterations 4  
time step continuity errors : sum local = 1.27091e-09, global = 1.82459e-19, cumulative = -3.02665e-20  
ExecutionTime = 0.106746 s ClockTime = 0 s
```

```
End
```

Um valor muito importante que aparece na tela é o **número de Courant**, definido como

$$C = \frac{\Delta t U}{\Delta x}$$

O número de Courant pode ser interpretado como uma razão entre a velocidade física e a velocidade numérica.

Para que a simulação seja estável, é interessante manter *Courant Number max* menor ou próximo de 1. Vamos voltar nesse assunto.

(Comentário: rode uma simulação em que  $C$  é maior que 1 e veja o que acontece.)

A simulação rodou. Várias pastas foram criadas. Cada pasta representa o resultado em um passo de tempo. O nome da pasta é o passo de tempo.

Em cada pasta temos um arquivo com o valor da velocidade ( $U$ ), da pressão ( $p$ ) e da vazão volumétrica nas superfícies ( $\phi$ ).

Além disso, temos também uma pasta, chamada *uniform*, com outras informações sobre o passo de tempo.

Essas informações permitem que iniciemos uma nova simulação a partir de qualquer instante de tempo.



Com o resultado da simulação, vamos começar o trabalho de **pós-processamento**.

Neste exemplo, vamos visualizar o resultado com o ParaView. Os próximos slides mostram alguns passos de como fazer essa visualização.

Se o ParaView não estiver aberto, execute, na pasta cavidadeCisalhante, o comando

**\$: paraFoam**

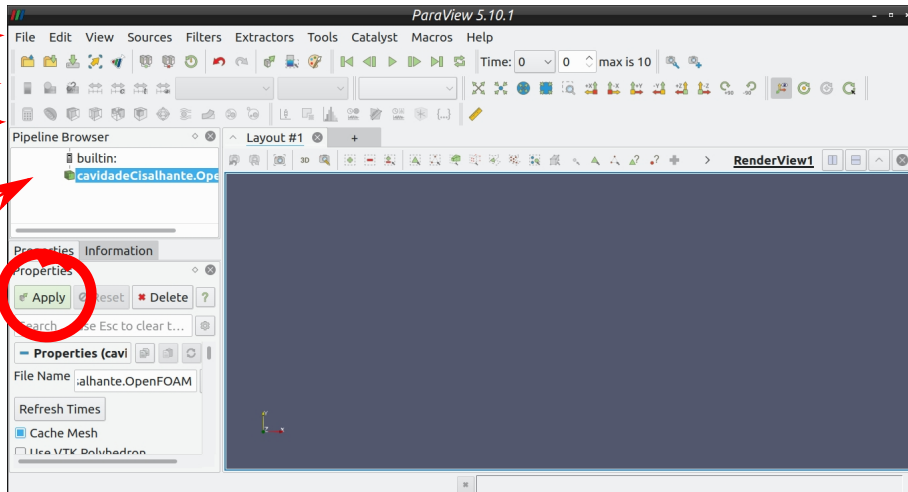
Clique em Apply.

Menu Bar →

Toolbars →

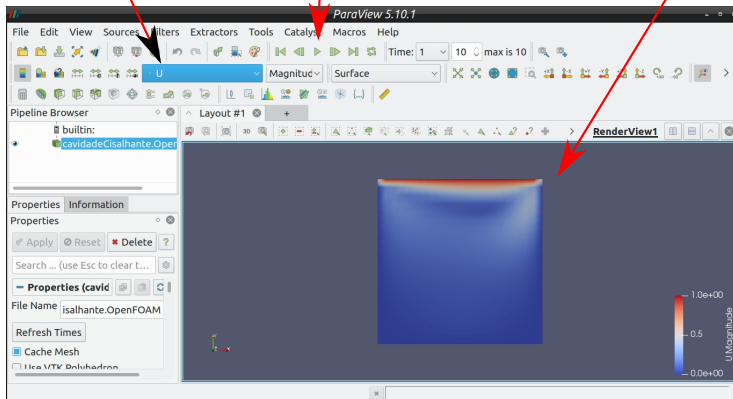
Filters →

Pipeline Tree →



Para visualizar o contorno de velocidade e de pressão.

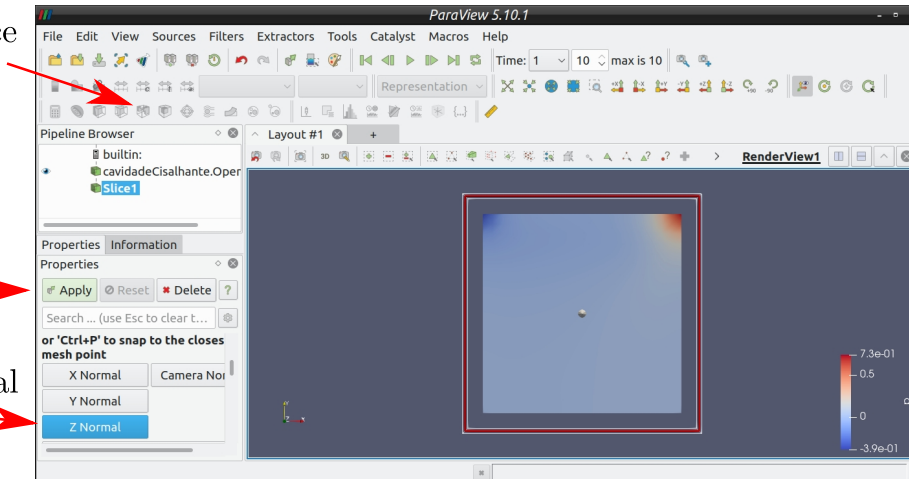
1. Escolha U
2. Aperte o Play
3. Use o mouse para mudar a câmera



4. Escolha p em vez de U e dê o play de novo.

Agora vamos visualizar as linhas de corrente do escoamento.

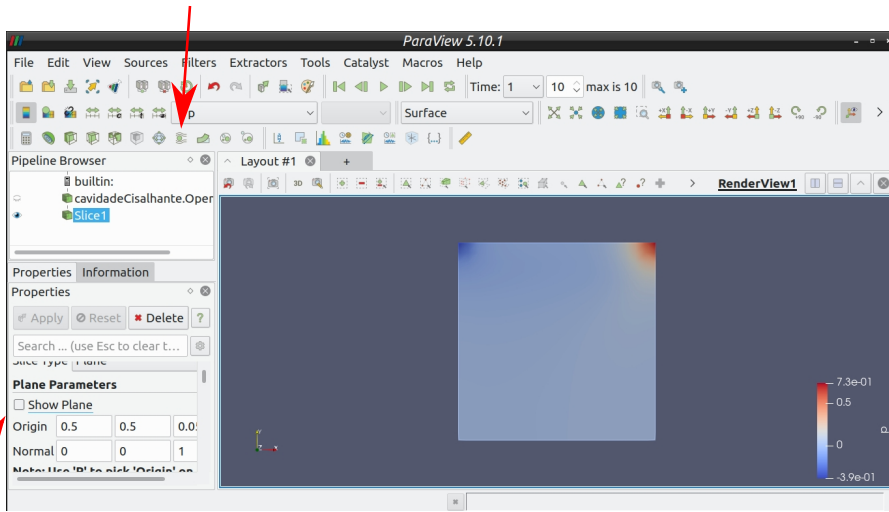
1. Slice



3. Apply

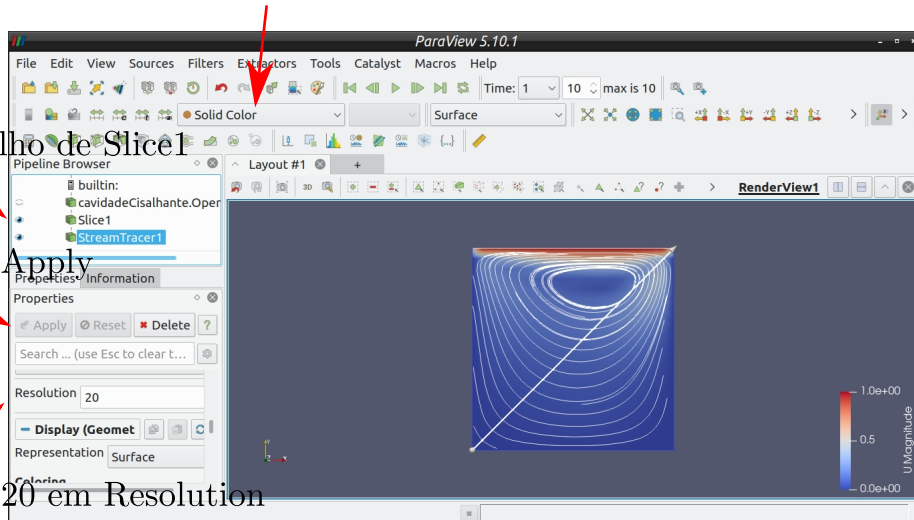
2. Z Normal

## 5. Clique em Stream Tracer



## 4. Desmarque a opção Show Plane

## 8. Escolha Solid Color para StreamTracer1



9. Clique no olho de Slice1

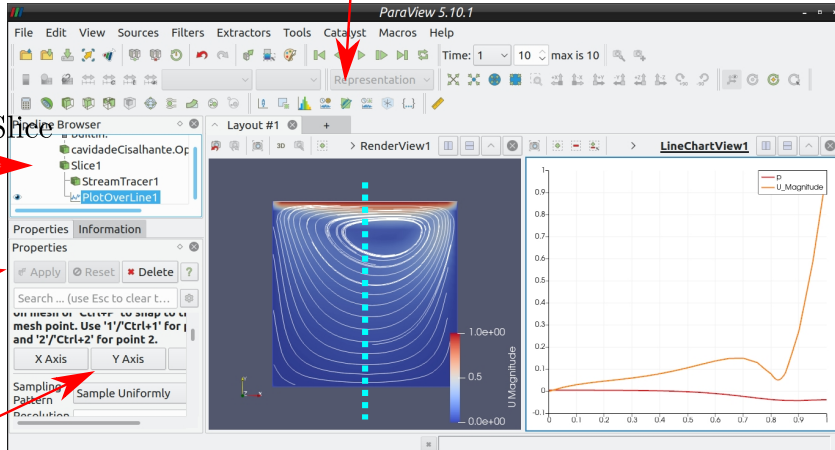
7. Clique em Apply

6. Coloque 20 em Resolution

Agora vamos plotar o valor de  $U$  e  $p$  ao longo de  $y$  no centro da cavidade.

## 2. Plot Over Line

1. Selecione Slice



4. Apply



3. Y Axis



Para exportar os resultados, vá em *File -> Save Screenshot ...*

Essa é apenas uma amostra do ParaView. O ParaView tem muito (MUITO) mais para oferecer. Vamos conhecer outras ferramentas ao longo do curso.

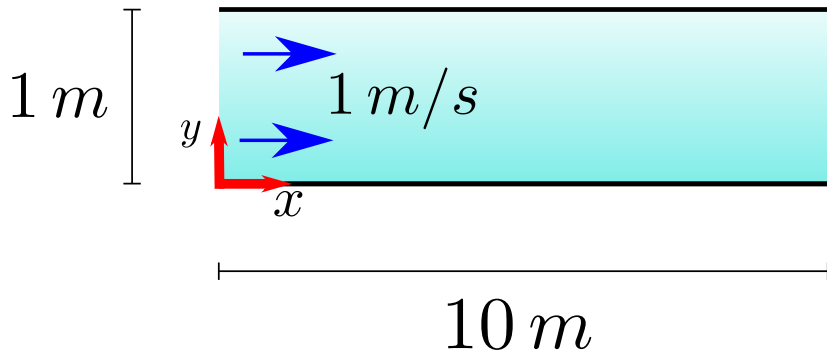
Também vamos ver como deixar os gráficos mais profissionais. Esse aí que fizemos agora está muito longe de ser publicável.



# Sumário

- 1 Cavityde Cisalhante
- 2 Escoamento Entre Placas
- 3 Cavityde Cisalhante 3D
- 4 Cavityde Cisalhante com Fluido Não Newtoniano
- 5 Conclusão

Vamos estudar agora o comportamento de um fluido newtoniano incompressível em um escoamento entre duas placas paralelas.



Agora temos entrada e saída de fluido.

Para começar, copie a pasta *cavidadeCisalhante* e dê o nome de *escoamentoEntrePlacas* para ela.

Rode o comando

```
$: foamCleanCase
```

Vamos ter que modificar o arquivo de geração de malha, o *blockMeshDict*, e os arquivos com as condições iniciais e de contorno, que são os arquivos *p* e *U* na pasta *0*.

Vamos deixar os outros arquivos da forma como estão.

Começando com o ***blockMeshDict***.

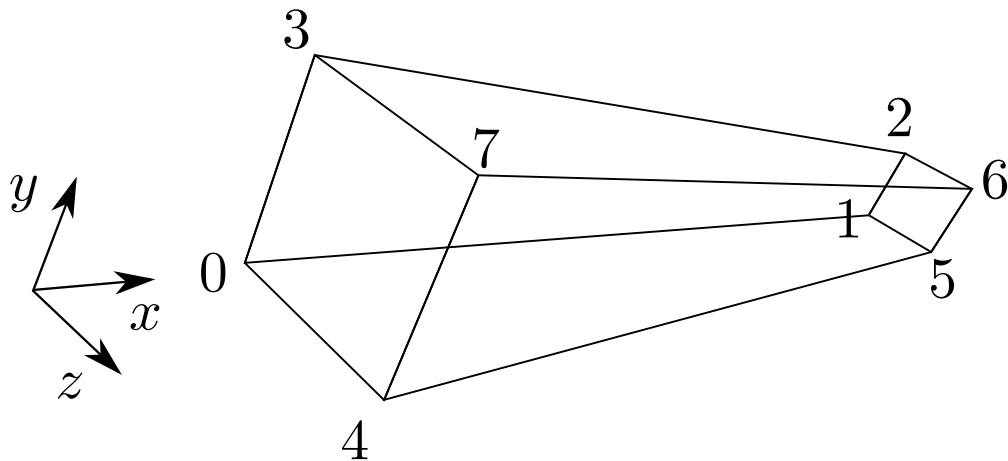
## *system/blockMeshDict*

```
1  convertToMeters 1.0;
2
3  vertices
4  (
5      (0 0 0)  // 0
6      (10 0 0) // 1
7      (10 1 0) // 2
8      (0 1 0)  // 3
9      (0 0 1)  // 4
10     (10 0 1)  // 5
11     (10 1 1)  // 6
12     (0 1 1)  // 7
13 );
14
15 blocks
16 (
17     hex (0 1 2 3 4 5 6 7) (100 10 1) simpleGrading (1 1 1)
18 );
```

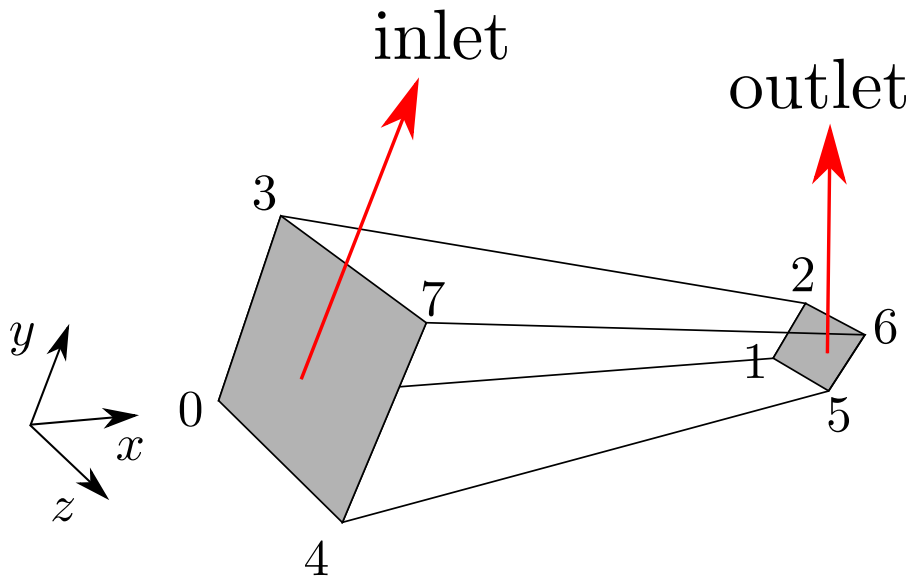
```
19 boundary
20 (
21     inlet
22     {
23         type patch;
24         faces
25         (
26             (0 4 7 3)
27         );
28     }
29     outlet
30     {
31         type patch;
32         faces
33         (
34             (2 6 5 1)
35         );
36     }
```

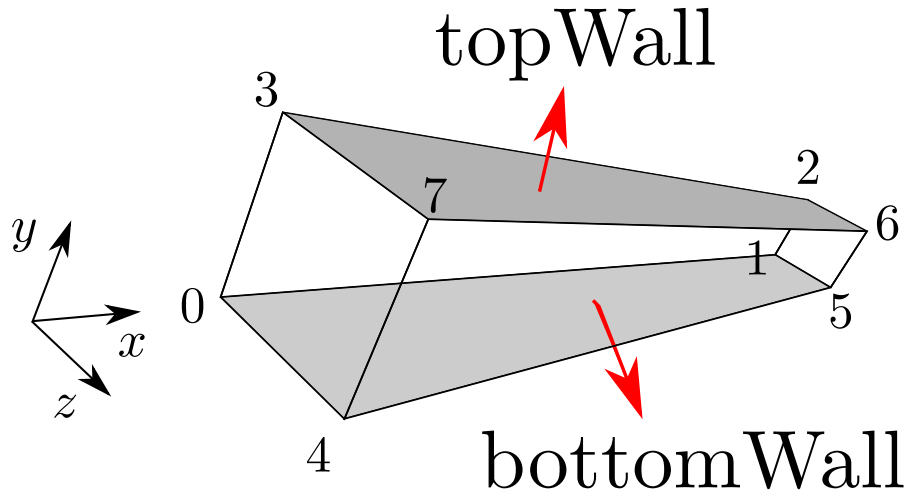
```
37 topWall
38 {
39     type wall;
40     faces
41     (
42         (3 7 6 2)
43     );
44 }
45 bottomWall
46 {
47     type wall;
48     faces
49     (
50         (1 5 4 0)
51     );
52 }
```

```
53     frontAndBack
54     {
55         type empty;
56         faces
57         (
58             (0 3 2 1)
59             (4 5 6 7)
60         );
61     }
62 );
```









$0/U$ 

```

1 dimensions      [0 1 -1 0 0 0 0];
2
3 internalField uniform (0 0 0);
4
5 boundaryField
6 {
7     inlet
8     {
9         type      fixedValue;
10        value      uniform (1 0 0);
11    }
12    outlet
13    {
14        type      zeroGradient;
15    }
16    topWall
17    {
18        type      noSlip;
19    }

```

```

1     bottomWall
2     {
3         type      noSlip;
4     }
5     frontAndBack
6     {
7         type      empty;
8     }
9 }

```

0/p

```

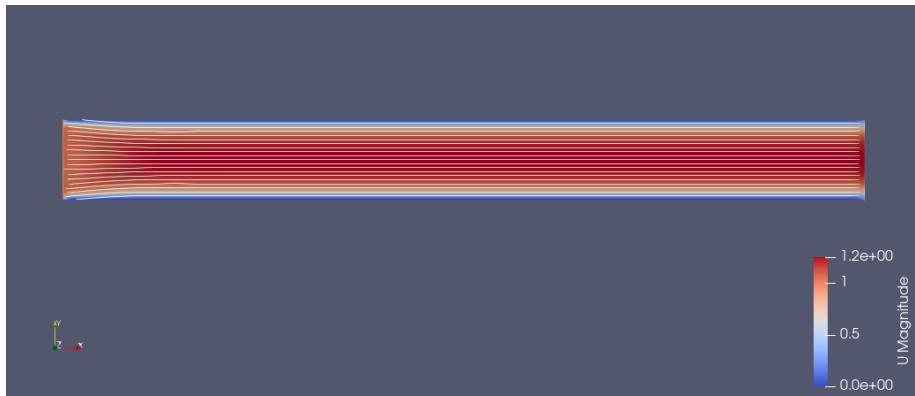
1 dimensions      [0 2 -2 0 0 0 0];
2
3 internalField uniform 0;
4
5 boundaryField
6 {
7     inlet
8     {
9         type          zeroGradient;
10    }
11    outlet
12    {
13        type          fixedValue;
14        value         uniform 0;
15    }
16    topWall
17    {
18        type          zeroGradient;
19    }

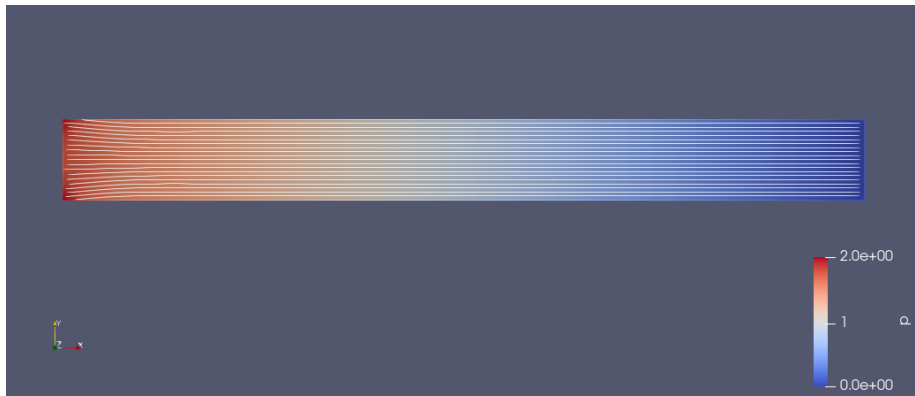
```

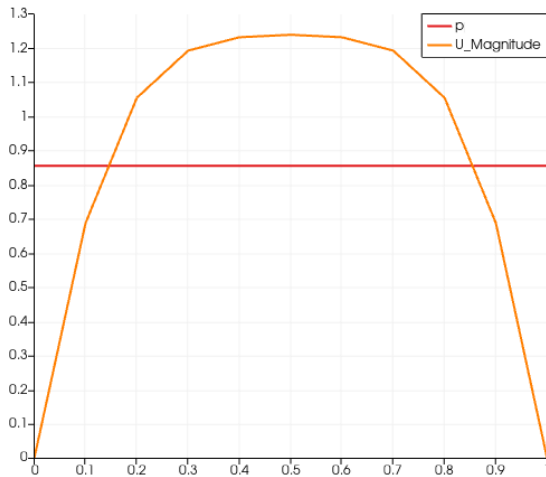
```

1     bottomWall
2     {
3         type          zeroGradient;
4     }
5     frontAndBack
6     {
7         type          empty;
8     }
9 }

```







# Sumário

- 1 Cavidade Cisalhante
- 2 Escoamento Entre Placas
- 3 Cavidade Cisalhante 3D
- 4 Cavidade Cisalhante com Fluido Não Newtoniano
- 5 Conclusão



Para transformar o nosso problema de Cavidade Cisalhante 2D em um problema 3D, basta adaptar as condições que usamos para as superfícies definidas como *frontAndBack*.

Primeiro, no *blockMeshDict*, coloque 20 pontos na direção  $z$  e modifique o tipo de condição de contorno de *empty* para *wall*.

No arquivo *0/U*, mude o tipo de *frontAndBack* para *noSlip*.

No arquivo *0/p*, mude o tipo de *frontAndBack* para *zeroGradient*.

Rode o caso e visualize o resultado.

# Sumário

- 1 Cavidade Cisalhante
- 2 Escoamento Entre Placas
- 3 Cavidade Cisalhante 3D
- 4 Cavidade Cisalhante com Fluido Não Newtoniano**
- 5 Conclusão

O OpenFOAM possui diversos modelos de **fluidos não newtonianos** implementados. Alguns exemplos são os modelos de Power Law, Cross Power Law, Bird-Carreau, Herschel-Bulkley e Casson.

Nesses casos, a viscosidade não é mais constante, mas depende da taxa de cisalhamento. No modelo Power Law, a viscosidade é calculada como

$$\nu = k\dot{\gamma}^{n-1}, \quad \text{para } \nu_{min} \leq \nu \leq \nu_{max},$$

em que  $k$ ,  $n$ ,  $\nu_{min}$  e  $\nu_{max}$  são parâmetros do modelo.

Para implementar o modelo Power Law no problema da cavidade cisalhante 2D, basta pegar o nosso caso e modificar o arquivo *momentumTransport* da forma como apresentado no próximo slide.

## *constant/momentumTransport*

```
1 simulationType laminar;
2
3 laminar
4 {
5     model          generalisedNewtonian;
6     viscosityModel powerLaw;
7
8     k              0.01;
9     n              0.5;
10    nuMax           0.1;
11    nuMin           0.01;
12 }
```

Rode este novo caso e veja o resultado.

Note que vai aparecer um arquivo a mais nas pastas com a solução de cada tempo. Esse arquivo a mais apresenta o valor da viscosidade em cada ponto do domínio. Agora a viscosidade depende da posição.

Visualize o escoamento e modifique os parâmetros do modelo Power Law para entender a influência deles no resultado.

# Sumário

- 1 Cavityde Cisalhante
- 2 Escoamento Entre Placas
- 3 Cavityde Cisalhante 3D
- 4 Cavityde Cisalhante com Fluido Não Newtoniano
- 5 Conclusão

Esse foi nosso primeiro passeio pelo OpenFOAM. Excelente!

Leva um tempinho até nos acostumarmos com esse novo mundo, mas aos poucos vamos entendendo como o OpenFOAM funciona.

Teste diferentes combinações de parâmetros e geometrias. Faça outras visualizações com o ParaView e explore outras ferramentas também.

É importante errar muito nessa fase de aprendizagem. É com esses erros que vamos desenvolvendo conexões e vamos entendendo as simulações.

Seu computador não vai explodir! (No máximo vai sair uma fumacinha, mas isso é tranquilo.)



Até a próxima aula.